Intelligence Never Grants Authority

A STAMP-Compliant Control Structure for Autonomous Agents

Jonathan R. M. Kershaw

Originator, Agent Responsibility Engineering

agentresponsibilityengineering.com | srexai.dev | 2026

Preprint (reconciled). Bounded STPA closure per `research/stpa/STPA_RESOLUTION.md`. **Public discipline materials** (README, tenets, PDFs): github.com/srex-dev/AgentResponsibilityEngineering. **This paper's evidence** is drawn from **privately held** implementation artifacts and a frozen bundle under `research/evidence-bundles/` — the **full monolith** is **not** asserted here as an open-source public clone. **Submission mechanics** (arXiv LaTeX vs Markdown, figures, where bundles live): `paper/ARXIV_AND_EVIDENCE_REALITY.md`. *Update commit hash when re-freezing.*

Abstract

Autonomous agent systems are increasingly deployed where **consequential actions** (API calls, data access, transactions) matter for safety, privacy, and compliance. Much of the prevailing stack is **observational or advisory**: prompts, filters, checkpoints, and logs that do not always sit on the **causal path** to side effects. This paper connects **Nancy Leveson's STAMP/STPA** tradition—safety as **control** of a **controlled process**, with feedback and an explicit process model—to **Agent Responsibility Engineering (ARE)**—a discipline with **public orientation** at [srex-dev/AgentResponsibilityEngineering](https://github.com/srex-dev/AgentResponsibilityEngineering), evaluated in this preprint against a **privately held** reference implementation (this manuscript describes **control structure and safety case**, not a turnkey public product drop).

We present a **bounded STAMP/STPA-complete** safety case **at a defined execution boundary**, not a proof of "safe AI" in the large. **Completeness** means: every hazard **in scope** in the working STPA package is **mitigated**, **accepted with documented residual risk**, or **assumption-bounded**, as recorded in the normative artifact `research/stpa/STPA_RESOLUTION.md`. The case is supported by **traceability** (losses, hazards, unsafe control actions, constraints), **component tests** (Rust and Go services on the golden path), an **interposition audit**, and a **frozen reviewer evidence bundle** (commit-attested). A **TLA+** specification skeleton states ordering obligations; it is **not** a machine-checked proof.

We **do not** claim universal fail-closed behavior in every subsystem, production validation in a named sector, or bidirectional formal proof of every narrative mapping in the broader ARE research program. We **do** claim: (1) a **STAMP-aligned** control reading of the governed path; (2) **causal interposition** (policy coprocessor → pre-execution validation → receipt binding →

executor); (3) **fail-closed** semantics for a **permit** verdict when the immutable **Ledger** write fails at the coprocessor (permit cannot leave without durable evidence—**Deny** with documented reason); (4) explicit **residual risk** and **out-of-scope** surfaces (deployment routing, strata composition, distributed lag, human escalation).

Keywords: STAMP, STPA, agent safety, execution boundary, causal interposition, governance, accountability, bounded safety case.

1. The Problem

Agents are in production. Not demos. Not pilots. Production systems making consequential decisions in regulated environments -- financial services, healthcare, critical infrastructure. They reason, plan, and act. They call APIs, read records, initiate transactions, and generate outputs that affect real people under real regulatory obligations. The governance approaches that exist were not designed for this. They were designed for a simpler version of the problem: a model that takes input and produces output, with a human reviewing the output before anything happens. That version of the problem is already obsolete. Prompt engineering constrains what the agent is told to do. It does not constrain what the agent can do. Output filtering catches some classes of bad outputs after the agent has acted. Human checkpoints insert review at defined decision points, but agents act faster than checkpoints can follow and the checkpoints are defined in advance of failure modes that will actually matter. None of these approaches provides what safety engineering has known for fifty years is required: a control structure. A control structure is not a rule set. It is not a monitoring system. It is not a policy document. A control structure is the complete system of controllers, controlled processes, control actions, and feedback mechanisms whose interaction produces safety as a property of the architecture. Safety engineering learned this by studying accidents. Accidents in complex systems do not happen because a component fails. They happen because the control structure failed -- because the controller was operating on a process model that had drifted from reality and issued actions that were wrong, or missing, or too late. Agent governance has not been analyzed through this lens. The failure modes that STAMP predicts for inadequate control structures are exactly the failure modes that ungoverned agent systems exhibit. The theoretical foundation has existed for fifty years. It has not been applied here. Agent Responsibility Engineering is the application. This paper gives a **bounded, evidence-aligned** account of that connection at the **defined execution boundary**—not a universal proof of safe AI.

2. STAMP: What It Says and Why It Applies

Nancy Leveson developed Systems-Theoretic Accident Model and Processes over decades of research into why complex sociotechnical systems fail. The core argument is a departure from how most engineers think about accidents. The traditional model says accidents happen because components fail. Find the broken component. Fix it. The model that replaced it in safety-critical engineering says accidents happen because of inadequate control. The system was not broken. The control structure failed to keep it safe. Safety is a control problem. Accidents result from a lack of appropriate constraints on the behavior of the system's components and their interactions. -- Leveson, Engineering a Safer World, 2011 This reframing has consequences. If accidents are

control failures, then safety engineering is the engineering of control structures. The question is not what broke but what the control structure failed to prevent, and why.

2.1 The Core Model

STAMP's analytical vocabulary is precise and small. A loss is an unacceptable outcome the system exists to prevent. A hazard is a system state that, given worst-case environmental conditions, leads to a loss. A safety constraint is a behavioral boundary whose enforcement prevents hazard state entry. The control structure is the complete system responsible for enforcing safety constraints. It has controllers that issue control actions, controlled processes that respond to them, and feedback mechanisms that return information about the controlled process state to the controller. Safety is a property of this structure, not of individual components. The process model is the controller's internal representation of the controlled process state. The controller does not act on reality. It acts on its model of reality. If the model is accurate, control actions will be appropriate. If it has drifted -- stale, incomplete, corrupted, or manipulated -- control actions will be wrong. Accidents follow. Keep the process model accurate. Everything else is in service of that.

2.2 The Feedback Requirement

The process model stays accurate through feedback. The controller issues a control action. The controlled process responds. Feedback returns information about that response to the controller, updating the process model before the next control action. This is a closed control loop. Open-loop control -- action without feedback -- cannot maintain process model accuracy over time. The model drifts. Actions become inappropriate. STAMP predicts this failure mode. It is not an edge case. It is the expected outcome of open-loop governance in a changing system. Most agent governance frameworks are open-loop. They constrain or monitor agent behavior but provide no mechanism for the governance system to update its model of what the agent is doing based on observed outcomes. The governance system issues constraints and then loses contact with the controlled process. STAMP predicts exactly what happens next.

2.3 Why STAMP Has Not Been Applied Here

STAMP was developed for physical controlled processes. Its implicit assumptions reflect that origin. The controller is distinct from the controlled process. The controlled process has a fixed identity determined by physical architecture. Feedback is generated by physical sensors the controlled process cannot manipulate. Autonomous agent systems violate all three assumptions. An agent can act as both controller and controlled process in nested workflows. Agent identity is not fixed -- it must be constructed and verified at runtime. An agent system generates its own records and can, without integrity controls, manipulate them. Prior STAMP applications to software-intensive systems exist -- primarily in aerospace and safetycritical embedded contexts. Applications to AI systems have focused on model-level safety analysis: examining individual inference pipelines, analyzing failure modes of specific components. No prior work uses STAMP as the theoretical foundation for the governance layer that sits above those components and

manages their actions in production. The violations of STAMP's physical assumptions do not invalidate the framework. They require its extension. The three extensions ARE provides are precisely targeted: the Passport for identity, the Ledger for manipulation-resistant feedback, the Coprocessor for physical interposition. Each addresses exactly one place where agent systems break STAMP's assumptions, and nothing else.

3. Agent Responsibility Engineering

Agent Responsibility Engineering is a governance discipline for autonomous agent systems in high-consequence environments. It was built to solve the production problem, not the theoretical one. The theoretical account came after. The governing principle: intelligence never grants authority. Authority is earned through verifiable policy enforcement. This is not a policy preference. It is an engineering requirement. An agent that is highly capable is not therefore trustworthy. Capability and authority are different properties. ARE treats them as different properties and provides a mechanism for granting authority that is independent of capability assessment. ARE is specified across four layers. Each layer corresponds to a distinct set of control structure responsibilities. The layers are not organizational. They are functional.

3.1 Foundational Layer: Identity and Authority

The Foundational layer answers a question STAMP assumes is already answered: who is the controlled entity, and what is it authorized to do. Every agent operating under ARE governance holds an Agent Passport: a cryptographic identity artifact issued at instantiation. The Passport encodes the agent's unique identifier, its delegated authority, its behavioral constraints, its operational scope, its chain of delegation from the issuing principal, and its expiry condition. It is signed by the issuing authority and verified at every policy evaluation. Without the Passport, the control structure cannot attribute actions to specific agents, cannot enforce per-agent authority boundaries, and cannot maintain an agent-specific process model. The Passport is what makes agents legible to the governance system. Legibility is a precondition for control. The Delegation Model governs how authority propagates through multi-agent workflows. A parent agent delegating to a child cannot grant authority it does not hold. The Passport encodes the delegation chain explicitly. The coprocessor enforces delegation boundaries at verdict time. Authority cannot be self-extended. The chain is verifiable and recorded.

3.2 Operational Layer: Real-Time Enforcement

The Operational layer is where governance becomes physical. The Governance Coprocessor sits between agent intent and execution. Every agent action must pass through it. No exceptions. When an agent proposes an action, the coprocessor intercepts it. It verifies the Passport. It queries the Operations Ledger for relevant process model state. It evaluates the proposed action against current policy. It issues a verdict: permit, modify, or block. The action proceeds only on permit or after modification. Block stops execution entirely and records the attempt. This is a STAMP controller in the precise technical sense. It maintains a process model. It issues control actions. It is physically interposed in the causal chain between the controlled process and the state change it

would produce. A governance system that can be bypassed is not a controller. It is a recommendation. The coprocessor closes that gap architecturally. The **Agent Responsibility Engineering** implementation is **not publicly distributed** as a turnkey product; this paper names the architectural roles (policy coprocessor, passport and registry services, pre-execution validator, atomic execution, receipts, immutable ledger) without publishing proprietary integration detail.

3.3 Epistemological Layer: Process Model and Learning

The Epistemological layer maintains the process model and drives learning from observed behavior. The Operations Ledger is the authoritative record of system state. Every agent action, every policy verdict, every state transition, every control intervention is recorded as an immutable, append-only hashchain event. The Ledger is the system's memory. It is also the feedback mechanism that keeps the controller's process model current and verifiable. The hashchain is a safety property, not an audit convenience. Each event is cryptographically bound to its predecessor. A falsified record breaks the chain. The manipulation is detectable. The Ledger is trust-minimized: it does not assume the agent is honest. It makes dishonesty visible.

3.4 Posture Layer: Control Structure Integrity

The Posture layer manages the control structure itself. It monitors the governance system for degradation. It manages promotion gates -- the criteria governing when an agent's authority can be expanded. It handles circuit-breaking when conditions exceed what the control structure can safely govern. Posture is the control structure's self-model. Most governance frameworks have no analog. They govern agents but have no mechanism for assessing whether the governance system itself is functioning correctly. Posture is that mechanism.

3.5 SDAVL: The Closed Control Loop

On the **documented golden execution path** (frozen evaluation configuration), ordering is enforced by the atomic execution engine: **validate → receipt → execute**, with the pre-execution validator binding a live passport and decision to the proposed action, and the policy coprocessor recording permit decisions under the **Allow+Ledger** rule (`STPA_RESOLUTION.md`, hazard **H6**). Broader closed-loop “Sense–Decide–Act–Verify–Learn” narratives in the source manuscript illustrate how STAMP’s feedback requirement can be elaborated in software; **this paper’s verified claims** are limited to what that **frozen configuration** tests and audits.

4. Bounded STPA closure and correspondence (execution boundary)

The earlier “Table 1” style mapping in the source manuscript expressed a **conceptual** alignment between STAMP vocabulary (loss, hazard, constraint, controller, feedback, process model) and ARE’s architectural intent. **This revision replaces any reading of that table as a machine-checked or universally complete formal proof.** Instead, we make two distinct claims, and we keep them separate:

1. **Correspondence (engineering narrative):** STAMP's control-structure reading applies to agent stacks when the **controlled process** is taken as "agents plus executors that touch the world," and the **controller** is the governance plane that issues permits, denials, modifications, and escalations. ARE instantiates that reading on a **golden execution path** using passports, a policy coprocessor, a pre-execution validator, receipts, atomic execution orchestration, and an append-only ledger—components named plainly here; **Appendix A** lists roles in **redacted** form for a public preprint.

2. **STPA completeness (within boundary):** For the **defined execution boundary** fixed in `research/stpa/STPA_PACKAGE.md` §1 and closed in `research/stpa/STPA_RESOLUTION.md`, every hazard **H1–H7** has exactly one disposition: **Mitigated**, **Accepted (bounded residual)**, or **Assumption-bounded**. That is what we mean by **bounded STAMP/STPA-complete**: **closure discipline** in the analysis layer, aligned to **test-backed** behavior on the path under review—not "all hazards in the universe eliminated."

Permit + Ledger (remediated invariant): On the policy coprocessor, a verdict that would **permit** execution must not leave the API unless the immutable **Ledger** write for that evaluation succeeds. Otherwise the effect is a **deny** with aligned evidence (`deny_reason: ledger_write_failed`). This closes a former gap where a permit could be returned without durable ledger evidence; see interposition audit scar tissue and Rust tests in the repository.

Broader narrative (SDAVL, predictive control, hypothesis validation): The source manuscript describes a Sense–Decide–Act–Verify–Learn loop, forward-looking evaluation, and ledger-grounded learning. Where those mechanisms are not fully exercised by the **current frozen snapshot**, they should be read as **research directions** or product roadmap—not as additional theorems checked in this paper's evidence bundle.

4.1 Scope boundaries (preserved intent)

What the **bounded** case covers: **enforcement of behavioral constraints at the execution boundary** for actions that flow through the governed coprocessor → pre-execution validation → receipt → executor chain, with the permit/ledger rule above.

What it **does not** claim: control over **internal** agent reasoning; **policy correctness** (the oracle); **emergent collective** dynamics across unbounded multi-agent networks without further analysis; **scope completeness** if operators expose **side channels** outside configured routing—those are **deployment assumptions** (`A_route`, `A_ex`) or explicitly **out of scope** per `STPA_RESOLUTION.md` §4–5.

4.2 Hazard-level closure (summary)

The authoritative table is `STPA_RESOLUTION.md` §1. In prose: **H1** (verdict bound to action) and **H4** (evidence before side effect) are **mitigated** by ordering and tests; **H2** is **mitigated** on the designed identity path; **H3** (revocation visibility) is **accepted** with a finite but not worst-case-bounded propagation window and fail-closed reads; **H5** (process-model drift) mixes **mitigation** and **accepted** residual; **H6** is **mitigated** for **Allow+Ledger** and **accepted** for certain **non-Allow** ledger failures outside the permit envelope; **H7** (strata disagreement) is **assumption / accepted**

outside the monolith. **Residual risk** (policy, humans, infra, multi-region ledger, recovery) is stated explicitly in that document—not minimized here.

5. The Agent Passport

STAMP's control structure model contains an implicit identity assumption: the controller knows what entity it is controlling. In physical systems this is satisfied by physical architecture. The controller is wired to specific actuators. The sensor reports from a specific location. Identity is not a runtime concern. In autonomous agent systems, identity is the hard problem. An agent has no physical presence. Its identity must be constructed, verified, and maintained at runtime. Without a mechanism for this, the control structure cannot attribute actions, cannot enforce per-agent authority, cannot maintain an agent-specific process model. The Passport is that mechanism.

5.1 What the Passport Contains

A valid Passport encodes: a unique cryptographic identifier for the agent, the issuing principal and full delegation chain, the scope of delegated authority, behavioral constraints inherited from the governance framework, operational boundaries, and an expiry condition. It is signed by the issuing authority and verified by the coprocessor at every verdict. Passport expiry is a safety mechanism, not an administrative one. An agent operating on a stale Passport is an agent whose authority state may no longer reflect current policy. From STAMP's perspective, this is process model inaccuracy -- the controller's model of the agent's authority does not match the agent's actual authorized scope. ARE treats Passport staleness as a hazard condition. Action is blocked pending re-issuance.

5.2 The Multi-Agent Recursion Problem

*Posture-layer meta-governance as described below is **design intent** in the broader ARE narrative; this paper's **verified** claims remain those traceable to the **frozen evaluation corpus** and `STPA_RESOLUTION.md`.*

A serious objection to any agent governance framework is the recursion question: who governs the governance agent When agents compose -- when a parent agent delegates to a child agent that may delegate further -- authority propagation becomes a primary attack surface. Three specific failure modes arise in multi-agent systems that the Passport and Delegation Model are designed to prevent. Authority amplification through composition : a child agent cannot be granted authority the parent does not hold. The Passport encodes the delegation chain. Every link in the chain is bounded by the issuing agent's own Passport scope. The coprocessor verifies the full chain at verdict time. An agent claiming authority it was not granted produces a chain that does not verify. Delegation of control logic : an agent cannot delegate the governance function itself. The coprocessor is not an agent. It is architectural infrastructure. An agent cannot instruct the coprocessor to issue verdicts differently on its behalf. The governance layer is not within the scope of what agents can control. Recursive controller problem : a **Posture** layer in the architecture narrative governs the governance system itself—monitoring the coprocessor, ledger write path, and passport registry for degradation. It does not resolve infinite regress; no finite

system can. Treat detailed posture semantics as **product and deployment** concerns unless tied to a specific test or gate in the evidence bundle. What the Passport and Delegation Model do not resolve: emergent authority accumulation across large agent networks operating over extended time horizons. That is identified as future work. Within the scope of first and second-order delegation -- which covers the vast majority of production multi-agent architectures -- the model is complete.

6. The Operations Ledger

STAMP's central safety concern is process model accuracy. The controller acts on its model. If the model is wrong, actions are wrong. Keeping the model accurate requires feedback that returns accurate information about the controlled process to the controller. For physical systems, that mechanism is sensors. For autonomous agent systems, sensors do not exist in the relevant sense. The agent generates its own activity records. A controlled process that can report on its own behavior can -- in the absence of integrity controls -- report inaccurately. The Operations Ledger is the answer. The Ledger is an append-only, cryptographically secured event log. Every agent action, every policy verdict, every state transition, every control intervention is recorded as an immutable hashchain event. It is simultaneously the system's memory, its audit substrate, and the feedback mechanism that keeps the controller's process model current and verifiable.

6.1 The Four Properties

Immutability means the historical process model cannot be corrupted or revised. The Ledger makes accuracy provable rather than asserted. What happened is what the Ledger says happened. Append-only means the Ledger captures temporal evolution of system state. STAMP's process model needs to reflect how the system arrived at its current state, not just where it is. The Ledger provides that history. Cryptographic integrity means the feedback loop is tamper-evident. Each event is bound to its predecessor by hash. A falsified record breaks the chain. Manipulation is detectable. Agent identity attribution means every Ledger event carries a Passport reference. Every recorded action is attributable to a specific, credentialed agent whose authority and history are verifiable. The process model is a record of who did what, under what authority, with what outcome.

6.2 Trust-Minimization, Not Absolute Trust

The Ledger does not claim to be unbreakable. It claims to make breaking detectable. This is the correct framing for a trust-minimized system and it matters for how the security properties should be understood. A reviewer will correctly identify that the Ledger's integrity guarantees depend on the integrity of the system writing to it. Three specific attack surfaces exist and should be stated explicitly. **Pre-Ledger actions:** an agent acting before durable evidence exists produces a gap in the record. On the **permit** path, the coprocessor does not return **Allow** without a successful ledger write for that evaluation (`STPA_RESOLUTION.md`, **H6**); on the **golden execution** path, receipt generation precedes executor invocation (**H4**). **Side-channel actions:** an agent that can act through channels outside the governed stack evades this safety case—**A_route** / **A_ex** and

interposition Option B in `STPA_RESOLUTION.md` §4. **Partial write failures:** behavior is **not** uniformly "halt the world": for **Allow+Ledger**, a failed write yields **Deny** with `ledger_write_failed`; for some **non-Allow** outcomes, a failed ledger write may still be returned and is **accepted** outside the *Permit* claim (same hazard table). Optional posture-style monitoring of chain continuity is **not** asserted here as universally implemented or fail-closed across every subsystem—the monograph language below should be read against that nuance.

6.3 Ledger as Feedback

In the **ideal closed-loop** narrative (Sense–Decide–Act–Verify–Learn), the Ledger closes the feedback channel: outcomes and verdicts inform the next decision. On the **paths this paper verifies**, governed consequential execution is paired with **durable** evidence—coprocessor **Allow** with ledger append, and atomic execution with receipt and outcome recording per `STPA_RESOLUTION.md` assumption **A_fb**. **Not** claimed: every Kafka topic, batch job, or sibling service produces a ledger event; only the **defined execution boundary** carries that obligation.

7. The Governance Coprocessor

Every component of ARE serves the control structure. The Passport makes agents legible. The Ledger makes the process model accurate and verifiable. On the **golden path**, **validate → receipt → execute** (and the **Allow+Ledger** rule) closes the enforcement loop the way STAMP expects a controller to bind control actions to feedback. The Governance Coprocessor is what makes the control structure binding at policy verdict time. STAMP distinguishes between a controller that recommends and a controller that enforces. A controller that can be bypassed is not a controller in STAMP's sense. It is documentation. Accidents happen when control actions are not applied. A governance framework that issues verdicts the agent can proceed without is not preventing accidents. It is recording them. The coprocessor sits between intent and execution. Nothing passes through without a verdict.

7.1 The Verdict Sequence

When an agent proposes an action, the coprocessor intercepts it before execution. It verifies the agent's Passport -- confirming identity, validating authority scope, checking expiry. It queries the Operations Ledger for current process model state: behavioral history, prior verdicts on similar actions, current system conditions relevant to policy evaluation. It evaluates the proposed action against the policy set encoded in the governance framework. It issues a verdict (including **permit**, **deny**, **escalate**, **unavailable**, and constrained modifications, per implementation). For exposition we often use **permit / modify / block**: **Permit** means the governed path may continue subject to downstream validation and receipts; **Block** stops the path; **Modify** narrows the action to a policy-safe form. The attempted action, the basis for blocking, and the agent's Passport state at the time of the attempt are recorded to the Ledger. Every verdict path produces a Ledger event. The process model updates regardless of outcome.

7.2 Physical Interposition

Physical interposition is what separates enforcement from monitoring. A monitoring system observes what happened and reports it. An enforcement system sits in the causal chain and determines what can happen. STAMP requires the latter. The coprocessor is architecturally interposed. It is not a sidecar that observes agent traffic. It is not a logging layer that records actions after they execute. It is a gate. The agent cannot reach execution without passing through it. The **Agent Responsibility Engineering** implementation is **not publicly distributed** as a turnkey product; this paper names the architectural roles (policy coprocessor, passport and registry services, pre-execution validator, atomic execution, receipts, immutable ledger) without publishing proprietary integration detail.

8. Failure Walkthrough: The Architecture Under Stress

Illustrative scenario for exposition—not a field incident report tied to the frozen evidence bundle.

Abstract claims benefit from a concrete walkthrough. The following **fictional but representative** trace shows how scope violation might be handled on the governed path: which components participate, and what durable evidence might record.

8.1 Scenario: Unauthorized data-access attempt (illustrative)

An agent in a **hypothetical** regulated-style workflow holds a Passport scoped to **read-only summaries**. It proposes an action to pull **full histories including sensitive fields**—beyond Passport scope (analogous to how sector rules would treat over-collection). **Step 1 — Request:** the action request hits the policy coprocessor; ledger and passport state may be queried for context. **Step 2 — Decide:** policy evaluation detects the scope delta; verdict is **deny/block** (not permit). **Step 3 — Act:** execution does not proceed on the governed path. **Step 4 — Evidence:** a verdict/outcome may be recorded per deployment wiring; **Step 5 — Learn:** subsequent evaluations can see prior denied attempts if recorded—exact escalation (circuit-break, HITL, revocation) is **policy and product** configuration, not asserted here for every build.

8.2 Escalation: Adaptive control under repeated violation (illustrative)

The source manuscript describes **tiered** responses (elevated block, posture flag, eventual passport revocation) driven by ledger history. That pattern matches how a **stateful** controller *could* tighten control when evidence accumulates. **Bounded claim:** repeated denied attempts must not silently **widen** authority without a governed re-issuance path; anything stronger (automatic revocation after *N* tries) is **policy/product**—verify against your deployment and tests before claiming it for a submission.

8.3 What this illustrates

Interposition wins when the path is real: a scope violation is caught **before** side effect if the request is actually forced through coprocessor + PEV + atomic ordering. **Evidence helps auditability** when verdicts and outcomes are durably recorded. **What is not proved by the story:** that every deployment enables the same escalation choreography, that regulators would accept a specific ledger schema, or that no human reviewer is ever required—those are **organizational** layers on top of the bounded technical case.

9. Lightweight Formalism

*The invariants below are **architectural intent** on the governed path; deployment must preserve interposition. They are not TLAPS theorems.*

The following formalism is intentionally minimal. Its purpose is to make the control structure's properties precise enough to reason about without requiring full formal verification. A complete temporal logic treatment is identified as future work in §15.1.

9.1 State, Action, and Control

Let S denote the system state, comprising the Ledger state L and the active Passport registry P: S = (L, P) Let A denote a proposed agent action. The control function C maps a system state and a proposed action to a verdict: C(S, A) -> { permit, modify(A'), block } Where modify(A') produces a modified action A' that is within policy. The control function is total: every (S, A) pair produces a verdict. There is no action that bypasses evaluation.

9.2 The Ledger Update Function

Let E denote a Ledger event. Each event is a tuple: E = (agent_id, action, verdict, policy_basis, hash(E_prev)) The Ledger update function U appends a new event to the Ledger, binding it to the previous state: U(L, E) -> L' where L' = L ++ E and E.hash = H(E_prev) Tamper detection: for any modified event E_m in the chain, H(E_m) != H(E_prev) for all subsequent events. The break is detectable at any subsequent verification step.

9.3 The SDAVL Loop as a State Transition System

The SDAVL loop can be expressed as a state transition system over S: Sense: query(L, agent_id) -> S_current Decide: C(S_current, A) -> verdict Act: enforce(verdict) -> outcome Verify: E = (agent_id, A, verdict, basis, H(L.head)) Learn: S' = (U(L, E), P) The system state after each loop iteration is S'. The next iteration operates on S'. The loop is closed: every action produces a state update that the next action evaluates against. The process model cannot become stale within a single action sequence because it is updated after every action.

9.4 The Safety Property

The central safety property of the architecture can be stated as: For all A: A executes -> exists verdict v in {permit, modify} such that C(S, A) = v Equivalently: no action executes without an affirmative policy verdict. This property holds if and only if the coprocessor is architecturally interposed on all paths to execution. The enforcement of this property is a deployment requirement. Its verification is a matter of architectural audit, not runtime checking.

9.5 The Audit-Completeness Invariant

Target invariant (governed consequential actions): for every action that **executes** on the golden path, there exists durable evidence tying attempt, verdict, and outcome—receipt and engine updates on the atomic path, and coprocessor ledger semantics for **permit** as in **H6** / **A_fb**. This is **not** "every subprocess in the fleet logs to one table," and it is **not** a TLAPS proof. It is the engineering shape the STPA package expects when we say feedback is real rather than decorative.

9.6 Evidence and explainability

The source manuscript states an **evidence-completeness** ideal: every verdict carries enough structured material for a reviewer to reconstruct **why** (not only **what**). In the implementation under evaluation, treat that as a **design target**: some paths expose rich decision payloads; others evolve release-to-release. **Do not** read §9.4–9.6 as lemmas proved for all verdict types and all deployments—read them as **compact specifications** of what "serious governance" should mean, checked where tests and the evidence bundle say they are checked.

10. The Complete Architecture

The four layers (identity, enforcement, process model, posture) describe a **unified** control story: safety properties emerge from interaction, not from a single component. **Within the execution boundary we analyze**, STAMP's roles map to named services (Appendix A); **outside** that boundary, the mapping is incomplete by construction. The Passport makes agents legible; the coprocessor binds verdicts; the ledger and receipts ground feedback; **SDAVL** remains a **narrative** label for "closed loop" thinking—not an extra formal system proved alongside STPA in this preprint.

10.1 The Interaction Sequence

On every agent action: the agent proposes an action. The coprocessor intercepts it. The coprocessor verifies the Passport -- identity confirmed, authority scoped, expiry checked. The coprocessor queries the Ledger -- process model state retrieved, behavioral history assessed. The coprocessor evaluates policy -- verdict issued. The verdict is enforced. The outcome is recorded to the Ledger as a hashchain event. The process model is updated. The loop closes. The next action request encounters a controller whose process model reflects verified current reality. That is what STAMP requires. That is what this architecture provides.

10.2 Failure Mode Analysis

A control structure that only works when nothing goes wrong is not a safety system. The architecture's behavior under failure conditions follows directly from the STAMP mapping and the formalism in Section 9. Ungoverned action : prevented by coprocessor physical interposition. The safety property in Section 9.4 holds: no action executes without a verdict. Incorrect policy verdict : mitigated by Ledger-informed process model. The process model reflects verified history, not agent-reported state. Passport staleness : treated as a hazard condition. Action blocked pending re-issuance. Ledger integrity failure: detectable with hash-chained stores when deployed. **Partial write / permit path:** coprocessor **Allow+Ledger** fail-closed per **H6**; other verdict classes may differ (`STPA_RESOLUTION.md`). Coprocessor bypass: an **out-of-scope** deployment failure unless routing and `A_route` hold. Each class should have an explicit disposition in operations playbooks; this paper does not claim every failure class is automatically circuit-broken in code.

11. Structural contrast with output-filtering guardrails

Qualitative comparison; NeMo Guardrails and ARE both evolve—check current docs before citing feature matrices.

The critical distinction is **state and placement**. NeMo Guardrails is stateless: it evaluates each output independently without reference to behavioral history. ARE's process model -- the Operations Ledger -- means every verdict is informed by what the agent has done across its entire operational history. A first-time scope violation and a tenth repeated scope violation look the same to NeMo Guardrails. To ARE they are categorically different events with different policy responses. The second critical distinction is interposition versus filtering. NeMo Guardrails operates after the agent has reasoned and produced an output. It filters what comes out. ARE's coprocessor operates before execution -- before the action reaches the system it would affect. Filtering catches outputs. Interposition prevents actions. In a regulated environment where the action is the violation -- accessing PII, initiating an unauthorized transaction -- filtering after the fact is not sufficient. The action must not execute. NeMo Guardrails does not claim to be a STAMP-compliant control structure. The comparison is not a criticism of its design goals. It is a clarification of what problem each framework solves. NeMo Guardrails constrains model outputs. ARE governs agent actions. These are different problems with different solutions.

12. Limits of ARE

A framework that claims to solve everything solves nothing. The following are the explicit boundaries of what ARE provides and where it does not apply. Stating these limits is not a concession. It is what makes the claims within the limits credible.

12.1 What ARE Does Not Govern

Agent internal reasoning : ARE governs what agents do, not what they think. The path from internal reasoning to proposed action is not observable. ARE's defense is at the action boundary.

An agent can reason incorrectly, unsafely, or adversarially -- and ARE will block the actions that result from that reasoning when they violate policy. But ARE does not prevent the reasoning itself. Policy correctness : ARE enforces policy without exception. It does not evaluate whether the policy is correct. A misconfigured policy enforced with perfect fidelity is still a misconfigured policy. Policy design is outside ARE's scope. Policy enforcement is ARE's entire scope. Policy generation is external to ARE and may be derived from regulatory frameworks, human governance boards, or automated synthesis systems operating on compliance requirements. ARE makes whatever policy exists unbypassable. It does not make policy right. Emergent collective behavior : ARE addresses individual agent governance and first and second-order delegation chains. Properties that emerge from the interaction of large agent collectives -- behaviors that no individual agent produces but that arise from aggregate action -are outside the current model. This is an active area of future work. Physical side channels : ARE assumes the coprocessor is the only path to consequential action. If an agent can act through channels outside the coprocessor's architectural scope, that action is outside ARE's governance boundary. Scope completeness is a deployment requirement that ARE does not enforce internally. The root of trust: ARE's cryptographic properties depend on the integrity of the key infrastructure underlying Passport issuance and Ledger signing. A compromised root of trust compromises the Passport and Ledger guarantees. ARE assumes the root of trust is secure. Securing it is an infrastructure requirement.

12.2 Where ARE fails closed (scoped)

Within the bounded case: missing or invalid passport/authority at the boundary tends to **deny/block** rather than silently permit; dependency read failures on the PEV path are treated **fail-closed** where designed; **Allow** without ledger append is **denied** at the coprocessor (**H6**). **Not claimed:** every auxiliary service halts the whole fleet on partial writes; **Escalate** (and similar) with ledger write failure can still appear in APIs and is **documented** as outside the *Permit* envelope. “Stop, record, escalate” is the **right operational posture** for many incidents—it is **not** a universal guarantee of every binary in the repo.

13. Evidence, reproducibility, and frozen packet

Public narrative (discipline, tenets, PDFs): github.com/srex-dev/AgentResponsibilityEngineering.

Research documentation spine (this repository): Normative STPA closure and traceability live under `research/stpa/` (index `research/stpa/README.md`): `STPA_PACKAGE.md`, `STPA_RESOLUTION.md`, `UCA_ENUMERATION.md`, `HAZARD_UCA_CONSTRAINT_TEST_CLOSURE.md`, `BOUNDARY_AND_ASSUMPTIONS_STPA.md`, `CAUSAL_SCENARIOS_SYSTEMATIC.md`. Interposition audit: `research/interposition_audit.md`. Claims and reviewer matrix: `research/claims_ledger.md`, `research/reviewer_attacks.md`. Track evidence summaries: `research/track-a-stamp/EVIDENCE_SUMMARY.md`, `research/track-b-

interposition/EVIDENCE_SUMMARY.md`; cross-track pointer: `research/DUAL_TRACK_SUMMARY.md`.

Implementation evidence for the claims in §4–§7 is drawn from component and integration tests on the **frozen** configuration captured in the evidence bundle—not as a substitute for deployment certification. Falsifiers include Rust and Go service tests on the golden path; lightweight Python harnesses additionally regress documentation and structure (exact harness layout may be omitted in a redacted public variant).

Frozen reviewer packets — procedure and layout: `research/evidence-bundles/README.md`. Submission snapshot `research/evidence-bundles/2026-04-26-stamp-safety-reviewer-packet-submission/` copies key research artifacts (`STPA_RESOLUTION.md`, `STPA_PACKAGE.md`, `claims_ledger.md`, `interposition_audit.md`, `SAFETY_CASE.md`, paper pointers), gate logs, `GIT_HEAD.txt` / `GIT_BRANCH.txt`, `MANIFEST.md`, and `FILES.sha256`. **Re-freeze** for a new submission by re-running `pytest tests/ -q`, `python tools/testing/run-all-internal-gates.py`, and the steps in `research/evidence-bundles/README.md`, then updating the commit hash in this paper's metadata.

Independent verification without full source: The frozen evidence bundle provides **verifiable artifacts** (cryptographic hashes, command transcripts, structured test and gate outputs, and normative STPA closure text) **sufficient for independent inspection of the paper's evidentiary claims**, even where the full implementation source is not publicly distributed.

Formal methods: `formal/tla/are_execution_boundary.tla` is a **TLA+ skeleton** expressing ordering obligations—it is a **proof target**, not a TLAPS proof.

14. Related Work

14.1 STAMP Applications in Software and AI Systems

STAMP has been applied extensively in aerospace, nuclear, medical devices, and transportation. NASA's adoption of STPA for launch vehicle safety analysis and the FAA's incorporation of STAMP-based analysis into avionics guidance represent the framework's deepest engineering adoptions. Leveson's later work increasingly addresses cyber-physical systems where software plays a central role in the control structure. Abdulkhaleq and Wagner (2015) developed XSTAMPP, establishing that STAMP's control structure model extends beyond purely physical processes. The framework is not inherently physical. It is inherently structural. This is the foundation for its application here. Applications to AI and machine learning exist but operate at a different level of abstraction. Salay, Queiroz, and Czarnecki (2018) apply STPA to machine learning components in autonomous vehicles, treating the ML subsystem as a controlled element within a physical control architecture. This work does not address the governance of autonomous agents acting independently across high-consequence decision boundaries. No prior work uses STAMP as the theoretical foundation for the governance layer above deployed agent systems.

14.2 Agent Governance Frameworks

The dominant agent governance approaches -- constraint-based, checkpoint-based, and monitoring-based -- share a structural failure when analyzed through STAMP. They are open-loop. They issue constraints or observe behavior but provide no mechanism for the governance system to update its model of what the agent is doing based on observed outcomes. An open-loop governance system cannot maintain process model accuracy. STAMP predicts the failure modes that result. NeMo Guardrails (NVIDIA, 2023) is a widely used toolkit for controllable LLM applications. **Section 11** gives a prose-only contrast (no maintained table in this Markdown). The recurring gap in STAMP terms: **output filtering** is not the same as an **interposed execution controller** with durable identity and path-level evidence. These are different solutions to overlapping but non-identical problems.

14.4 AI Safety and Alignment

The alignment and interpretability research community addresses a related but distinct problem: ensuring that AI systems behave in accordance with human values and intentions. Constitutional AI (Bai et al., 2022), RLHF (Christiano et al., 2017), and interpretability work operate at the model layer. The distinction is this: alignment asks whether the model will choose to behave safely. ARE asks whether the system will permit unsafe behavior regardless of what the model chooses. Safety engineering's answer to this question has been consistent for fifty years: do not rely on the controlled process choosing to stay within safe boundaries. Build a control structure that enforces boundaries regardless. ARE is that control structure.

15. Conclusion

Leveson's STAMP/STPA tradition gives agent governance a **control-structure** vocabulary: controllers, controlled processes, feedback, process models, and explicit hazards—not only component failure. **Agent Responsibility Engineering** implements a **governed execution path** where permits are bound to identity and policy state, receipts precede side effects on the golden chain, and **permit plus durable ledger evidence** fail closed together.

The contribution of this **revised** preprint is **honest closure**: a **bounded STPA-complete** disposition record for hazards **H1–H7** at the **defined execution boundary**, with **traceability** to tests and audits, and with **out-of-scope** and **residual** items stated plainly. That is a **systems safety case** contribution—not a certificate of universal safe AI.

Intelligence never grants authority. On the path we analyze, **authority** is separated from **capability**: it is enforced through verifiable gates and recorded evidence, under assumptions operators must still honor in deployment.

15.1 Future work

Machine-checked proofs of ordering and ledger semantics; tighter treatment of multi-agent emergence; expanded related work on runtime policy and cryptographic governance; and unified metrics for long-lived delegation lineages remain active directions—some are sketched in the source manuscript's future-work paragraphs and are **not** claimed closed here.

Additional references (from source manuscript)

Rabanser, S., Kapoor, S., Kirgis, P., Liu, K., Utpala, S., & Narayanan, A. (2026). Towards a Science of AI Agent Reliability. arXiv:2602.16666. Princeton University.

Fernandez, M. (2026). Agent Control Protocol: Admission Control for Agent Actions. arXiv:2603.18829. TraslaIA.

Kaptein, M., Khan, V.-J., & Podstavnychy, A. (2026). Runtime Governance for AI Agents: Policies on Paths. arXiv:2603.16586.

Mazzocchetti, A. M. (2026). Cryptographic Runtime Governance for Autonomous AI Systems: The Aegis Architecture for Verifiable Policy Enforcement. arXiv:2603.16938.

Gaurav, S., Heikkonen, J., & Chaudhary, J. (2025). Governance-as-a-Service: A Multi-Agent Framework for AI System Compliance and Policy Enforcement. arXiv:2508.18765.

Abdulkhaleq, A., & Wagner, S. (2015). XSTAMPP: An eXtensible STAMP Platform as Tool Support for Safety Engineering. STAMP Workshop, MIT.

Bai, Y., Jones, A., Ndousse, K., et al. (2022). Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. arXiv:2204.05862.

Christiano, P., Leike, J., Brown, T., et al. (2017). Deep Reinforcement Learning from Human Preferences. NeurIPS 2017.

Federal Reserve Board. (2011). SR 11-7: Guidance on Model Risk Management. Board of Governors of the Federal Reserve System.

Kershaw, J. R. M. (2025). Guardian-Agent (historical coprocessor reference implementation). *Superseded for this safety case by the later ARE architecture; no public distribution URL.*

Kershaw, J. R. M. (2026). Agent Responsibility Engineering: Tenets and Foundational Framework. *Public materials:* github.com/srex-dev/AgentResponsibilityEngineering. *(Monolith implementation underlying this safety case is evaluated from internal frozen artifacts; verify venue disclosure policy.)*

Leveson, N. (1995). Safeware: System Safety and Computers. Addison-Wesley.

Leveson, N. (2011). Engineering a Safer World: Systems Thinking Applied to Safety. MIT Press.

Leveson, N., & Thomas, J. (2018). STPA Handbook. MIT Partnership for Systems Approaches to Safety and Security.

NIST. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0). National Institute of Standards and Technology.

NVIDIA. (2023). NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications. github.com/NVIDIA/NeMo-Guardrails.

Salay, R., Queiroz, R., & Czarnecki, K. (2018). An Analysis of ISO 26262: Using Machine Learning Safely in Automotive Software. SAE Technical Paper.

Thomas, J. (2013). Extending and Automating a Systems-Theoretic Hazard Analysis for Requirements Generation and Analysis. MIT PhD Thesis.

Core references (STPA / systems)

1. Leveson, N.G. *Engineering a Safer World: Systems Thinking Applied to Safety.* MIT Press, 2012.

2. Leveson, N.G. and Thomas, J.P. *STPA Handbook.* MIT, 2018.

3. Saltzer, J.H., Reed, D.P. and Clark, D.D. End-to-End Arguments in System Design. *ACM TOCS*, 2(4):277–288, 1984.

4. Schneider, F.B. Enforceable Security Policies. *ACM TISSEC*, 3(1):30–50, 2000.

5. Lamport, L. *Specifying Systems: The TLA+ Language and Tools.* Addison-Wesley, 2002.

6. NIST. *Artificial Intelligence Risk Management Framework (AI RMF 1.0).* NIST AI 100-1, 2023.

7. Agent Responsibility Engineering — public discipline repo (README, PDFs): github.com/srex-dev/AgentResponsibilityEngineering. **Monolith** evidence in this preprint: internal repository paths and frozen bundles — **not** asserted as a full public source clone.

8. Normative STPA closure and evidence spine (this repository): `research/stpa/README.md`, `research/stpa/STPA_RESOLUTION.md`, `research/stpa/STPA_PACKAGE.md`, `research/stpa/UCA_ENUMERATION.md`, `research/interposition_audit.md`, `research/evidence-bundles/README.md`.

Additional citations from the source manuscript (Rabanser et al., runtime governance preprints, NeMo Guardrails, XSTAMPP, etc.) may be merged for arXiv camera-ready; keep **claim–citation alignment** so citations do not imply evidence the bundle does not contain.

Appendix A. Implementation roles (public preprint — topology redacted)

Role	Public preprint naming
Policy evaluation / coprocessor	Policy coprocessor service
Identity / passports / delegation	Passport issuance, agent registry, authority engine
Pre-execution gate	Pre-execution validator
Orchestrated execution	Atomic execution engine
Evidence binding	Receipt generator
Durable history	Immutable ledger
Edge transport	API gateway (transport only)

*A **reviewer supplement** may map roles to internal paths under confidentiality.*

Appendix B. Hazard closure at a glance (normative summary)

Authoritative rationale, residuals, and assumptions: `research/stpa/STPA_RESOLUTION.md` §1.

ID	Hazard (abbrev.)	Resolution (within boundary)
H1	Execution without verdict bound to action	**Mitigated**
H2	Invalid / forged / expired authority as valid	**Mitigated**
H3	Revocation not visible in time	**Accepted** (bounded)
H4	Evidence not bound before side effect	**Mitigated**
H5	Stale / incomplete process model	**Mitigated** + **Accepted** (residual)
H6	Degraded control / Ledger leaves permit path	**Mitigated** (Allow+Ledger) + **Accepted** (non-Allow edge)
H7	Strata disagree; pipeline not fail-safe	**Assumption** + **Accepted** (outside monolith)

Appendix C. Figure (text): golden governed execution chain

Single-chain causal order asserted for **verified** claims (see §3.5, §4, frozen tests):

proposed action

policy coprocessor (verdict; Allow requires Ledger write — H6)

pre-execution validator (live passport / binding)

receipt generator (evidence binding)

atomic execution engine (validate → receipt → execute)

executor (side effect)

Not a circuit diagram; deployment routing and side channels are explicit assumptions in `STPA_RESOLUTION.md`.

Version note: Assembled by `tools/paper/build_stamp_arxiv_reconciled.py` from `paper/STAMP_ARE_docx_paragraphs.txt` plus reconciled inserts; §5.2–§12.2 and §14.2 are spliced from `paper/stamp_arxiv_alignment_block.md` and `paper/stamp_arxiv_alignment_14_2.md` when present. Submission layout vs LaTeX: `paper/ARXIV_AND_EVIDENCE_REALITY.md`. Re-freeze evidence and update the commit hash when preparing submission.